PSAI_104

SUBSTRATE MOVE SYSTEM PALLET (PART 2)

- **Team Name:** Eiger
- **Payment Address:** Fiat 14.04.2023, 16:50 UTC+3
- **Level:** 3

PROJECT OVERVIEW ****🗒****

This application is a response to **Move Smart Contract Pallet RFP** and a follow-up to our previous work on the RFP.

OVERVIEW

> 🕮 Terminology: **Substrate Move System Pallet** - A Substrate system pallet that can be used as a building block for substrate-based chains, it exposes interfaces to interact with the Move virtual machine. **Substrate Move** - A Move language fork that is Substrate compatible.

- The goal is to provide a Substrate system pallet that allows to deploy and interact with Smart Contracts written in the Move language, by providing a Move Virtual Machine (MoveVM) as a pallet.
- The project directly improves the growth potential of the Substrate based ecosystem by providing support for one of the most modern smart contract programming languages and VM types out there - Move.
- We are interested in creating this project because we are firm believers in the modular vision of web3, and only by collaborative efforts on improving and unifying the technology between different chains and the languages, will we get better products as an outcome.

Goal - Level up the growth possibilities of the Substrate ecosystem by providing a way to develop and execute Move smart contracts on Substrate.

This is the second phase of a 3-phase development plan:

1. In-Depth Exploration and Assessment of MoveVM and Substrate Integration
2. **MoveVM compatibility work and Subtrate Pallet development**
3. Finalising the Substrate-Compatible MoveVM

PROJECT DETAILS

Prior work

We are basing the core architecture and many of the design decisions on the Pontem networks developed system pallet for Move VM. They had maintained their own fork of the Diem Move language, which was used as the base execution layer for their version. Both of these repositories have not been maintained for a very long time already.

♻ We aren't seeking to maintain any of the existing codebase; rather, we aim for a full revival through a new greenfield project. Our rationale for this stems from the substantial advancements made in the Rust, Substrate, and Move ecosystems since Pontem's latest commits from over a year ago. We believe that handling potential code rot due to the passage of time might be more labor-intensive than starting afresh and drawing upon existing projects for more current guidelines.

Documentation of core components, architecture

1. **Substrate Move:**

   The first part of the project will be a MoveVM fork, as some major changes will need to be made to the codebase for it to be substrate compatible. For example:

   - no_std compatibility: Making it lightweight and suitable for use in Substrate runtimes.
   - wasm32 target compatibility: Adapting all the VM's different components to work efficiently and securely on the wasm32 target architecture.

   We created the fork of Move VM in a manner that allow us and the community to easily follow and track changes from the upstream, thus making the maintenance and change tracking to be much simpler.

2. **Move VM system pallet:**

   The second part of the project will be a Substrate virtual machine pallet in Substrate. This is a modular component that is needed to integrate a specific new VM into a Substrate runtime. It will serve as a bridge between the runtime and the Move VM, managing resources and translating data or actions between the two environments.

API specifications

As a minimum, we plan on providing all of the RPC calls that the Pontem crate did. As the team progresses with the implementation, we might add or remove RPC calls as we best see fit.

Move language has a concept of "gas" for executing contracts, whereas Polkadot uses "Weights". Each Move transaction invocation requires providing a gas limit for execution, and it's necessary to be able to transform the values between weight and gas:

- mvm_gasToWeight
- mvm_weightToGas

Estimating gas for different operations:

- mvm_estimateGasPublish
- mvm_estimateGasExecute

Working with the primitives of the Move language:

- mvm_getResource
- mvm_getModuleABI
- mvm_getModule

Tech stack

We plan on using Rust for developing the system pallets and using existing Move language smart contracts for end-to-end testing of the whole workflow.

Notes

Because the Move language requires a fork to work with Substrate chains, and modifications to the address size, there might be incompatibilities with deploying existing Move Smart Contracts from other chains that make use of the address properties size, as well as the compiled ABI and bytecode for those contracts might be invalid. To deploy on our MoveVM system pallet, the forked toolchain must be used to re-compile all smart contracts. The address size of Move language is configurable via a feature switch with 32-bits being one of the options.

ECOSYSTEM FIT

Move is a smart contract programming language that emphasizes access control and scarcity, offering some unique advantages over other popular VMs in blockchain ecosystems.

The importance of bringing the MoveVM to Polkadot was recognized over two years ago when Pontem Network started working on a Move virtual machine pallet to execute Move smart contracts on Substrate-based chains. Although they discontinued the project and haven't updated the codebase for over a year, the W3F still keeps this RFP, which serves as evidence that porting the MoveVM is crucial for the future of the Polkadot network.

We concur with this perspective and have actually been actively researching the MoveVM - exactly with a similar idea of helping port it over to other chains.

TEAM 👥

TEAM MEMBERS

- **Karlo Mardešić** (GitHub, LinkedIn) is a Software Engineer at Eiger and has experience with telecommunications and low-level drivers in C/C++. These days his expertise has shifted to blockchain technology and P2P protocols, where he primarily uses Rust to tackle exciting problems.
- **Piotr Olszewski** (GitHub, LinkedIn) is a Software Engineer at Eiger, and has over 13 years of professional experience, with a strong academic background in distributed computing. He has a large bag of experiences, ranging from military appliances, cryptographic projects, telecommunication software to embedded platforms. During his career, Piotr took different roles, from developer to team and tech leader. His main tools are C/C++ and Rust.

CONTACT

- **Contact Name:** Daren Tuzi
- **Contact Email:** daren@eiger.co (Founder & CEO of Eiger)
- **Website:** Eiger.co

LEGAL STRUCTURE

- **Registered Address:** Linnankatu 3 A 24, 20100 Turku, Finland
- **Registered Legal Entity:** Eiger Oy****

TEAM'S EXPERIENCE

Web3 promises to upgrade the very foundations of our society — from money, finance, and governance to media, gaming, and science. To deliver on that promise, decentralised technologies are to be integrated into the everyday experiences of billions of people. For engineering, this is a mountain of a challenge.

Eiger was founded to develop infrastructure for web3 mass adoption. We help technology companies improve and integrate the core technologies of web3 to meet the climbing demands for scaling and performance.

We currently employ 25+ senior web3 engineers across the globe and work with some of the most ambitious organisations in the industry, including Forte, Aleo, and XRP Labs, to name a few.

Eiger is part of the Equiilibrium group. We have been working under the Equilibrium brand for the past 4 years and only last year created the Eiger brand to focus on implementation engineering. Eiger/EQ was one of the main contributors in all of these endeavours.

TEAM CODE REPOS

As mentioned in the Teams section, Eiger already has extensive experience developing large infrastructural projects. Some chosen examples:

- The SnarkOS for Aleo, the privacy-focused smart contract L1 platform going into production this summer. https://github.com/AleoHQ/snarkOS
- OrbitDB is a distributed, p2p, serverless database. https://github.com/orbitdb/orbit-db
- The rust implementation of IPFS, now archived. https://github.com/rs-ipfs/rust-ipfs

- The main implementation of Interledger in Rust, which we developed and maintained. https://github.com/interledger/interledger-rs
- Ziggurat - A Network protocol testing framework for ZCash, XRP, and Algorand. Notably, critical network vulnerabilities were found and reported to the core teams. https://github.com/runziggurat

DEVELOPMENT STATUS 📖

- This application is a direct response to this RFP
- Move language Github
- Pontem MoveVM fork
- Pontem MoveVM system pallet
- MoveVM pallet GitHub contains our work so far

DEVELOPMENT ROADMAP 🔩

OVERVIEW

- **Total Estimated Duration:** 2 months
- **Full-Time Equivalent (FTE):** 2 FTE
- **Total Costs:** 96 000 USD
- **Starting Date:** 04/09/23

MILESTONE 1 - MOVEVM COMPATIBILITY WORK AND SUBTRATE PALLET DEVELOPMENT

- **Estimated Duration:** 2 months
- **FTE:** 2
- **Costs:** 96 000 USD

Goal: Create a customised Move VM solution for the Substrate ecosystem, using the knowledge gained during the first milestone. The deliverable will be a pallet capable of receiving, storing and executing Move smart contracts.

Number	Deliverable	Specification
0a.	License	Apache 2.0 and MIT
0b.	Documentation	We will provide both inline documentation of the code and a basic tutorial that explains how a user can (for example) spin up one of our Substrate nodes and send test transactions, which will show how the new functionality works.
0c.	Testing and Testing Guide	Core functions will be fully covered by comprehensive unit tests to ensure functionality and robustness. In the guide, we will describe how to run these tests.
0d.	Docker	We will provide a Dockerfile(s) that can be used to test all the functionality delivered with this milestone.
1.	Rust crate: Substrate Move	Forking the Move VM, as deemed necessary from the research done during the first milestone. The alterations will include everything to create the virtual machine Substrate-compatible. We will also provide extensive documentation of how the whole process is designed and how it is to be maintained.
2.	System Pallet: Substrate Move SP adds Move functionality	Integrating the Move VM runtime within the custom pallet, ensuring compatibility with the Substrate blockchain and Move smart contract execution.
3.	System Pallet: Substrate Move SP APIs to interact with the Move VM	Developing an API that enables developers to interact with the Move VM on Substrate-based chains, allowing them to deploy, execute, and manage Move smart contracts.

FUTURE PLANS

This is the second phase of a 3-phase development program:

1. In-Depth Exploration and Assessment of MoveVM and Substrate Integration
2. **MoveVM compatibility work and Subtrate Pallet development**
3. Finalising the Substrate-Compatible MoveVM

The **next** step will be to submit a grant proposal to finalize this work - finishing up the pallet and making sure it works properly and can be utilized by other developers .

We hope that upon the completion of all phases of creating the Substrate Move System Pallet, it will open doors for further collaboration and community input on the project. We strive to have the codebase well documented so that others might join in and contribute.

While there are no long-term plans set in stone for the usage of this pallet, we have had discussions about possibly creating a parachain, possibly a common good parachain, that utilizes this MoveVM implementation and would run MoveVM contracts. As we near the completion of this initial development, we will be discussing these future plans more in-depth..

ADDITIONAL INFORMATION +

How did you hear about the Grants Program?

We learned about it when looking at open RFPs by the web3 foundation on their website.

We wanted to get back up to date on what is happening in the Polkadot ecosystem, and working on grants, specifically RFPs, has been a great way to do so.

Looking to apply to other RFPs currently open as well. Stay tuned!